Chapter 10: Working with Data

Many careful thinkers, when beginning a new project, seek out multiple archives for research. Katherine Bode, for example, argues for the importance of scholars building their own resources as a means of challenging the prevailing reliance on canonical texts and curated corpora, as solely engaging canonical texts risks reinforcing narrow or dominant histories.[1] A plurality of archives and texts offers an opportunity to think critically about the boundaries and biases of each archive under consideration. Many of the approaches to text mining we have profiled already – especially distinctiveness and distance measures – offer tools for systematically extending the study of the boundaries of the archive, such that a scholar can accumulate critical perspectives as they broaden their access to archives.

When a scholar is pursuing a complex research question, the relevant data must often be located, extracted, and prepared independently before analysis can begin. In historical research or other research problems across the social sciences and humanities, relevant sources may be scattered across different archives and exist only in incomplete or inconsistent formats. While basic guidance can provide some degree of comfort to the analyst who wishes to gain access to the diverse kinds of data available on the web, today's data ecosystem is highly diverse and even fragmented in terms of its infrastructures.

Data diversity brings new technical challenges. The method of extracting and analyzing data vary with how the data is stored and processed. Some archives offer well-documented APIs that facilitate access to records, while others provide only partial documentation or inconsistent metadata. The differences in how data is stored often reflect underlying disparities in the resources allocated to developing, maintaining, and preserving digital archives. Very few archives come packaged as a software repository as `hansardr` is. Therefore most data analysts will eventually want to engage the broader data ecosystem that exists beyond the data prepared for this book.

As Ian Milligan argues in *History in the Age of Abundance*, the explosion of born-digital content has transformed the historian's archive, requiring new tools and methods to access, process, and interpret digital records at scale.[2] In fields ranging from the humanities to computer science, analysts have documented approaches that are useful for aggregating text from the web.[3] Once data has been properly accessed, cleaned, and stored, the analysis exercises from the rest of the book can be applied towards a variety of research questions – but only if the analyst is able to manipulate data.

The skills of accessing and manipulating data from the web are not well distributed. Our own experience suggests that, far from being a familiar task to most college seniors with a degree in data science, downloading and processing data from the web is often beyond the abilities of most students. Even computer scientists graduates skilled in data processing may face trouble when working with unfamiliar file types. Few textbooks offer a practical approach for analysts who wish to meet a growing digital archive, and the existing how-to guides published by historians and archives are, while useful, often disaggregated. Other The piecemeal approach to accessing one digital archive at a time rarely arms the analyst with the skills they need to feel comfortable ingesting large amounts of data.

To access the broader data ecosystem, analysts need to understand at least three modes of accessing data:
(a) access to APIs, (b) bulk downloads (common on sites like Hathitrust or Harvard Dataverse), and
(c) the topic of web scraping and extracting data through RSS feeds

In this chapter, we show how historians can engage with data housed in online archives–even though the data freely available may be found in disparate and irreconciliable types, many of which are increasingly born-digital.

The ability to engage fragmented infrastructures under any circumstance provides the key to becoming flexible enough to adopt the many archives that may been needed for a grand project.

In the age of AI, many analysts compile their own archives or have collected data from primary sources, resulting in a haphazard and sometimes poorly-labeled dataset that needs much cleaning before it is fit for analysis. Even when the data has been carefully treated by librarians or other custodians, the datasets rarely arrive in a neatly curated or structured state.

In result, analysts need a conceptual framework that enables them to identify and collect data for their archive, organize their data, and inform how they address missing data.

To handle the data with grace and ease, analysts also need to be aware of two further concepts: (i) the cleaning of data, (ii) data formats (how data is stored), data types (what kinds of data are being represented), and data structures (how data is organized for use and analysis),

Our hope is to guide readers' imaginations without limiting them. The concepts we introduce in this chapter are intended to be applicable across a wide range of historical research domains–whether in traditional historical scholarship, cultural heritage studies, or fields like the digital humanities or public history.

Through a series of exercises, the following chapter will introduce high-level concepts and best practices that will put the analyst on the way to data fluency.

Throughout this chapter, we will introduce several new libraries as they become relevant. We found it easier to explain each library alongside the analysis steps that use it. However, because `tidyverse` and `tidytext` are used throughout the chapter, and we are already familiar with `hansardr`, `knitr`, and `quanteda` from previous chapters, we'll load them now.

```
library(hansardr)
library(knitr)
library(quanteda)
```

```
## Package version: 4.3.1
## Unicode version: 14.0
## ICU version: 70.1
```

```
## Parallel computing: disabled
```

```
## See https://quanteda.io for tutorials and examples.
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.1
## v purrr     1.2.1
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidytext)
```

APIs

Although vast quantities of cultural and historical materials are now available online through APIs (Application Programming Interfaces, or mechanisms that allow one piece of software to request and receive data from another), the ability to actually retrieve and work with APIs remains unevenly distributed. Accessibility matters because an increasing share of cultural production is "born digital," meaning the digital record is the original form rather than a later digitized copy.[4]

In our experience, collecting data from the web is not a routine skill — even among graduating seniors in data-science programs. Computer science students who are otherwise proficient at data wrangling can still struggle when confronted with unfamiliar file formats, download endpoints, or web-based collection workflows.

Part of the problem is that very few instructional materials teach the practices of working with APIs in a systematic way. Several works exist that outline the history of APIs, conceptualize them as a scholarly artifact, or discuss their significance to digital scholarship.[5] However, they do not demonstrate how to use an API.

Most textbooks treat data as if it were already neatly stored on a local machine, while archivists' how-to guides tend to focus on individual platforms one at a time without offering a transferable workflow. A specialized approach may leave analysts adept at navigating a single repository but unprepared to ingest larger, more heterogeneous collections of digital sources at scale. Lessons on accessing one digital archive at a time rarely arms the analyst with the skills they need to feel comfortable ingesting large amounts of data from diverse sources.[6]

Creating a Dataset from an API Endpoint

An API is a set of rules and protocols that allows one piece of software to interact with another. An API endpoint is a specific web address that a program can use to access data stored on another server. Analysts can connect to an API endpoint in two main ways: by sending standard HTTP requests directly from an R script, or by using an R package designed to interact with the API, which simplifies the connection and data retrieval process. This section will demonstrate both approaches.

Direct Connection to an API using an HTTP Request

Increasingly, public institutions and government agencies have adopted open data initiatives that make datasets accessible through APIs. Many cities, such as the City of Dallas and New York City, provide access to a range of datasets—such as 311 service requests, building permits, and police reports—via their open data portals.

APIs are a central mechanism through which analysts can access the vast quantities of data available online. API endpoints function not merely as technical tools, but as gateways into contemporary digital archives. Large-scale datasets—ranging from open government records to social media content—are often made accessible through API endpoints, which return data in structured formats such as CSV, or more commonly, JSON and XML.

Successfully interacting with open APIs—such as those provided by the City of Dallas or New York City—often depends on navigating several technical considerations, including user authentication, rate limits on the number of requests, options for customizing queries, and the specific data formats returned by the API. In what follows, we provide an overview of these considerations.

Sample APIs

Some examples of API endpoints include those offered by Reddit or The Guardian. Reddit, for instance, allows users to retrieve the latest posts in a given subreddit using an endpoint like:

`https://www.reddit.com/r/datascience/new.json`

This URL can be accessed programmatically using R, or the analyst can manually inspect the data by copying and pasting it into a web browser's address bar. When accessed, this URL returns a structured list of recent submissions to the /r/datascience Subreddit in JSON format.[7]

Similarly, *The Guardian*, a global news platform, offers an API that enables users to search for news articles on a given topic, such as education, via a URL like:

`https://content.guardianapis.com/search?q=education`.

Using an API: Setting up an API Key

Using *The Guardian's* API requires additional set up. Accessing many APIs–particularly those provided by commercial services or large-scale platforms like *The Gaurdian* requires user authentication in the form of an API key.

An API key is a unique string of characters that identifies the user making the request. API keys are often used to monitor usage and enforce security. To obtain an API key, users typically need to register with the data provider. For example, to use *The Guardian*'s API, one must register and generate a key at:

`https://open-platform.theguardian.com`.[8]

Once obtained, the key is appended to API requests—for example:

`https://content.guardianapis.com/search?q=education&api-key=your-api-key-here`.

Using an API Key: Authentification

Authentication is the process by which a user verifies their identity to the API, typically by including an API key with each request for data. Importantly, once an API key tethered to a user is submitted, the interaction is no longer anonymous—the exchange of information is now linked to a specific account, allowing the API provider to monitor usage, enforce rate limits, or restrict access if necessary. In this chapter, we will not use an API that requires authentication.

Practical Exercise: The API in Action

To illustrate how an API works, consider the following example from the New York City Open Data portal, "NYC OpenData", which provides access to vast public datasets including population data, budgetary data, and data on city infrastructure. For this exercise we will look at just current and historic 311 service requests. The link below takes us to a webpage for the dataset *311 Call Center Inquiry*, which also provides a direct link to the API endpoint.

`https://data.cityofnewyork.us/City-Government/311-Call-Center-Inquiry/wewp-mm3p/about_data`.

Copying and pasting the API endpoint from this page,

`https://data.cityofnewyork.us/resource/wewp-mm3p.json`,

into a web browser will return nested data—typically in JSON format—which can be expanded to reveal the individual fields and their associated values. In this dataset, each entry includes the date and time the request was made, the agency responsible for handling the request, the name of the request, and a brief description. We will now read this data into the R environment for data processing and analysis.

In the following code, we use two new R packages: `httr` for sending HTTP requests, and `jsonlite` for parsing JSON data into a tidy data frame that mirrors the data structure we have been processing throughout the book.[9]

The code sets the user up for a conversation with a server, with the following components of the dialogue: * A "request" is a message sent from a client (in this case, your R script) to a server, asking for specific information or resources—such as a dataset. * The "result" is stored in the "response" object, which represents the message returned by the server. A response includes a status code (e.g., 200 for success, 404 for not found), headers (which contain metadata), and the body (which holds the requested content, such as the JSON data containing the service request data).

Please note the following components of the API call: * The `GET()` function from the `httr` package sends an HTTP `GET` request to the specified API endpoint to retrieve data. * The `status_code()` function checks whether the request was successful. If it was, `content()` extracts the raw JSON text from the response body. * We use the `fromJSON()` from `jsonlite` to convert the data received from the API from JSON format a data frame. * The `flatten = TRUE` argument is necessary for simplifying any nested JSON structures into flat, readable columns suitable for analysis in R.

```
library(httr)
library(jsonlite)

# Define the JSON API endpoint
url <- "https://data.cityofnewyork.us/resource/wewp-mm3p.json"

# Make GET request to extract JSON content
response <- GET(url)

# Check whether the request was successful
# - status_code() returns the HTTP status code
# - a status code of 200 indicates a successful request
if (status_code(response) == 200) {

  # Extract the JSON content as text
  json_content <- content(response, "text", encoding = "UTF-8") # Read the JSON response

  # Convert the JSON into a tibble
  service_requests <- fromJSON(json_content, flatten = TRUE) %>% # Parse the JSON into a data frame
    as_tibble() } else {
  stop("Failed to retrieve data. Status code: ", status_code(response)) } # Stop if the request failed
```

Because this request was successful, we now have a data frame containing the shelter data:

```
# Show just the first five columns
head(service_requests[, 1:5])
```

```
## # A tibble: 6 x 5
##   unique_id date                   time        date_time                agency
##   <chr>     <chr>                  <chr>       <chr>                    <chr>
## 1 105394645 2014-12-12T00:00:00.000 2:39:49 AM  2014-12-12T02:39:49.000 NYCSERV~
## 2 105394646 2014-12-12T00:00:00.000 9:45:08 AM  2014-12-12T09:45:08.000 DOE
## 3 105394647 2014-12-12T00:00:00.000 11:30:33 AM 2014-12-12T11:30:33.000 DOF
## 4 105394648 2014-12-12T00:00:00.000 3:08:08 PM  2014-12-12T15:08:08.000 HRA
## 5 105394649 2014-12-12T00:00:00.000 12:50:33 PM 2014-12-12T12:50:33.000 HPD
## 6 105394650 2014-12-12T00:00:00.000 12:44:44 PM 2014-12-12T12:44:44.000 OLR
```

Examining the Data Returned by APIs: JSON Records

APIs typically return data in structured formats such as JSON (JavaScript Object Notation), XML (eXtensible Markup Language), or CSV (Comma-Separated Values). JSON is widely used because of its interpretability across programming languages.

The service request API returned data in the JSON format. Using the `fromJSON()` function, and we converted the JSON to a tabular format for readability.

For the sake of understanding the data transformations that occurred, we also kept a copy of the nested JSON data. Many programmers favor JSON because it strikes a strong balance between human readability, machine parsability, and broad compatibility. JSON is the default for most modern APIs.

The window below gives an example of what that JSON data looks. Note that JSON uses key-value pairs, bracketed collections of information called "arrays", and .

Clicking on the `json_content` data in the Global Environment panel will also provide the means to explore the JSON data.


```
## [
##     {
##         "unique_id": "105394645",
##         "date": "2014-12-12T00:00:00.000",
##         "time": "2:39:49 AM",
##         "date_time": "2014-12-12T02:39:49.000",
##         "agency": "NYCSERVICE",
##         "agency_name": "NYC Service",
##         "inquiry_name": "NYCService Donations",
##         "brief_description": "Information about in-kind food donations.",
##         "call_resolution": "Agency Closed"
##     },
##     {
##         "unique_id": "105394646",
##         "date": "2014-12-12T00:00:00.000",
##         "time": "9:45:08 AM",
##         "date_time": "2014-12-12T09:45:08.000",
##         "agency": "DOE",
##         "agency_name": "Department of Education",
##         "inquiry_name": "Pre-Kindergarten - Apply - Half Day",
##         "brief_description": "Information about pre-kindergarten programs in New York City.",
##         "call_resolution": "Information Provided"
##     }
## ]
```


When we made this request, we were only able to obtain 1,000 records (which is represented by 1,000 rows, where each row corresponds with a single record).

```
# Count the number of rows in the dataset
nrow(json_list)
```

```
## [1] 1000
```

However, if we return to the dataset's page —

`https://data.cityofnewyork.us/City-Government/311-Call-Center-Inquiry/wewp-mm3p/about_data`

—

We can see that the site reports the dataset contains millions of records. Technically, nothing went wrong during data extraction. Instead, we were "rate limited." Rate limiting is a common mechanism used by open data APIs to manage server load and prevent abuse. It restricts how many requests a user can make within a given time period. For example, unauthenticated users might be limited to 100 requests per hour, while authenticated users may be allowed significantly more—such as 10,000 requests per hour. Even though the data is publicly available, access can still be throttled based on usage patterns and credentials.

Most platforms open government data websites using the Socrata Open Data API (SODA) limit responses to 1,000 records per request by default. If a dataset exceeds the request limit, only the first 1,000 rows are returned unless pagination is used. In the following example we revise the above code to account for pagination. This code runs two cycles, collecting up to 1,000 records per cycle for a total of up to 2,000 records. This code can be modified to extract additional records.

```
# Define the JSON API endpoint
url <- "https://data.cityofnewyork.us/resource/wewp-mm3p.json"

# Set the number of records to retrieve per request
limit <- 1000

# Create an empty tibble to store the collected data
service_requests <- tibble()

# Retrieve the first two pages of results
# - offset = 0 retrieves rows 1-1000
# - offset = 1000 retrieves rows 1001-2000
for (i in 0:1) {
  offset <- i * limit # Calculate the starting row for this request

  # Make a paginated request
  response <- GET(url, query = list(`$limit` = limit, `$offset` = offset)) # Request one page of data

  # Check whether the request was successful
  if (status_code(response) != 200) {
    stop("Request failed. Status code: ", status_code(response)) } # Stop if the request failed

  # Parse the JSON response
  data_subset <- fromJSON(content(response, "text"), flatten = TRUE) # Convert the JSON into a data fra

  # Add the new records to the existing data
  service_requests <- bind_rows(service_requests, as_tibble(data_subset)) } # Append the new rows
```

Importantly, even if you use pagination, you may still violate a website's Terms of Service. Some websites explicitly require delays—such as adding a sleep interval between requests—to avoid overloading their servers with requests that exceed typical human usage.

Using R Packages to Connect to an API

The examples above demonstrate basic interactions with APIs that require only a single, fixed URL. However, many APIs–especially those offering more extensive datasets–support more complex queries. APIs require us to construct URLs, which can be a nontrivial task. Challenges stem from constructnig URLs that often include multiple parameters, such as search terms, filters, authentication keys, and pagination controls, all of which must be formatted correctly to retrieve the desired data. After the data has been via the API, it still needs to be parsed into an R-friendly format.

Numerous R packages have been developed to simply interactions with APIs. These R packages encapsulate the API's functionality within R functions and data structures, abstracting away the more complex tasks such as URL construction and parsing JSON responses. For example, the rOpenGov universe–composed of a community that develops open-source R packages to increase access to government data–offers several packages that simplify working with APIs, including `usdoj` and `oldbailey`. This section demonstrates the use of `usdoj`.[10]

`usdoj` provides more intuitive access to data from the U.S. Department of Justice's API by wrapping complex API interactions into simple R functions. `usdoj` can be installed directly from the rOpenGov universe.

```
# Load the usdoj library
library(usdoj)
```

We can now use `usdoj` functions to collect data from the U.S. Department of Justice. The following code will extract just 10 blog entries from the U.S. Department of Justice. Specifying the search direction as `DESC` instructs `usdoj` to extract the most recent blog entries.

```
# Retrieve the 10 most recent Department of Justice blog entries
# - n_results specifies how many blog entries to retrieve
# - search_direction = "DESC" returns the newest entries first
blog_entries <- doj_blog_entries(n_results = 10, # Retrieve 10 blog entries
                                 search_direction = "DESC") # Sort from newest to oldest
head(blog_entries$teaser, 5)
```

```
## [1] "On January 19, ICITAP Counter-IED Advisors delivered Burkina Faso's Groupe Spéciale d'Interventi
## [2] "The U.S. Supreme Court declined to review a decision by the U.S. Court of Appeals for the Sixth
## [3] "On January 13, ICITAP Albania joined the U.S. Embassy Tirana and international law enforcement p
## [4] "On January 6, ICITAP marked its 40th anniversary - four decades of global engagement united by c
## [5] "Secretary of War Pete Hegseth has announced bold action to prioritize religious liberty and
```

From this point, it is easy to analyze words.[11]

```
# Find the ten most common words in the blog entries
# - select() keeps only the blog text
# - unnest_tokens() splits the text into individual words
# - anti_join() removes common stop words
# - count() counts the occurrences of each word
# - slice_max() keeps the ten most frequent words
top_words <- blog_entries %>%
  select(body) %>% # Keep the blog text
  unnest_tokens(word, body) %>% # Tokenize the text into words
  anti_join(stop_words, by = "word") %>% # Remove common stop words
  count(word, sort = TRUE) %>% # Count the occurrences of each word
  slice_max(n, n = 10) # Keep the ten most frequent words

# Visualize the most common words
# - aes() maps words and word counts to the plot
# - geom_bar() creates a bar chart
# - coord_flip() flips the axes for readability
# - labs() adds the title and axis labels
# - theme_minimal() applies a minimal theme
ggplot(top_words,
       aes(x = reorder(word, n), # Order words by frequency
```

```
          y = n)) + # Plot word counts
geom_bar(stat = "identity") + # Create the bars
coord_flip() + # Flip the axes
labs(title = "Top 10 Words in DOJ Blog Entries", # Add the chart title
     x = "Word", # Label the x-axis
     y = "Frequency") + # Label the y-axis
theme_minimal() # Apply a minimal theme
```

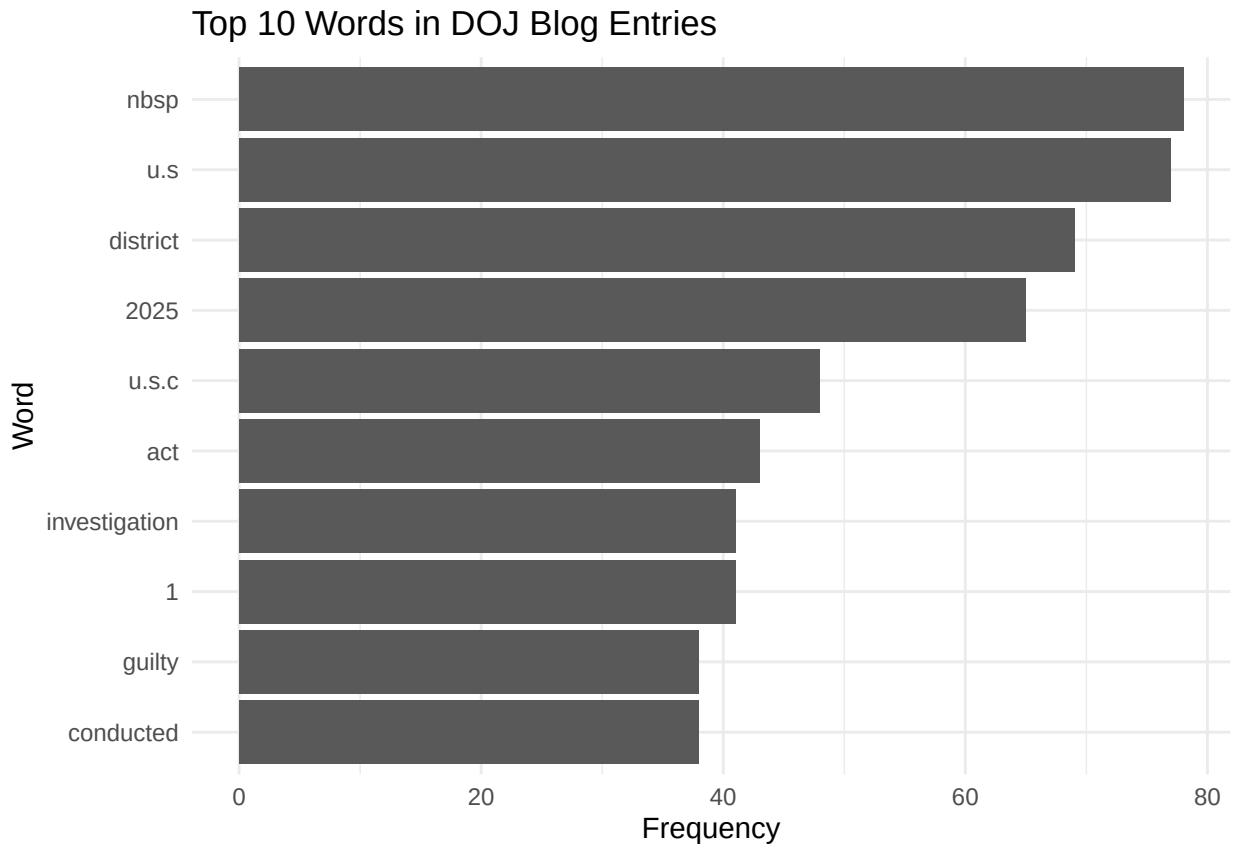


Figure 1: Top 10 words in United States Department of Justice Blog Entries, retrieved on November 24th, 2025.

APIs for Scholarly Research

Here we present a list of APIs for scholarly researcher derived from the University of Virginia Library with description provided by the authors of this book.[12]

Source	Description
AM Digital	Adam Matthew Digital is a major publisher of digitized primary source archives used in humanities and social science research. The company partners with libraries, archives, and museums to digitize rare historical materials—such as manuscripts, letters, maps, colonial records, newspapers, and ephemera—and curates them into themed online collections for scholarly use.
Coherent Digital	Coherent Digital is a U.S.-based digital publisher and aggregator that specializes in building digital collections of "hidden, lost, and endangered materials" — grey literature, non-traditional sources, web-only or at-risk content, and materials from underrepresented communities (especially in the Global South).
Dewey Data	Dewey Data is a platform designed to provide access to curated datasets from third-party premium providers that have traditionally been difficult for scholars to obtain. It aggregates people, place, and company data—such as foot-traffic, real estate, spending, web analytics, and points of interest. By negotiating licensing and integration on behalf of researchers, the platform makes commercially controlled datasets more accessible for scholarly work under academic-friendly terms.
Edinburgh University Press	Edinburgh University Press supports text and data mining through a licensed crawler-based system that provides institutional researchers with access to full-text XML of books, chapters, and journal articles for non-commercial academic use. It does not offer a public-facing API but rather a web crawler that once enabled, researchers can systematically harvest the XML corpus, making the platform suitable for large-scale linguistic, historical, or computational text analysis while maintaining compliance with publisher licensing terms.
Elsevier	Elsevier provides a suite of research APIs that give programmatic access to citation data, metadata, abstracts, and in some cases full text from its major platforms, including Scopus, ScienceDirect, Reaxys, PharmaPendium, and Engineering Village. These APIs support a wide range of scholarly workflows, from bibliometrics and literature reviews to chemical data retrieval and research evaluation. Most APIs are available at no cost for non-commercial academic users, provided their institution already subscribes to the underlying products.
HathiTrust	HathiTrust is a large-scale digital library that provides access to millions of digitized books and archival materials from research libraries, with strong coverage of public-domain works and U.S. government documents. For text and data mining, it offers the HathiTrust Research Center (HTRC), which provides computational access to full text through curated datasets and a Data Capsule environment that allows large-scale analysis while protecting copyrighted material. Researchers can download public-domain corpora directly or use controlled infrastructure to analyze in-copyright texts, making HathiTrust one of the most important platforms for scholarly text mining in the humanities and social sciences.

Source	Description
IEEE Xplore	Institute of Electrical and Electronics Engineers (IEEE) offers the IEEE Xplore API, a developer toolkit that enables programmatic access to its digital library of engineering, computer science, and technology content. Through the API users can query metadata, abstracts and, in certain cases, full-text articles. Access requires registration and an API key. It supports automated workflows like automated literature discovery and citation analysis.
New York Times API	The New York Times API is a set of publicly documented web APIs that allow developers and researchers to access structured data from the NYT, including articles, metadata, archives, books, reviews, and topical tags. It provides access to the Article Search API (historical and current articles with metadata), Books API (bestsellers and reviews), Archive API (full monthly issue metadata back to 1851), Movie Reviews API, and Top Stories API. Most endpoints return JSON and are designed for metadata-level access rather than full article text, though headlines, abstracts, keywords, and URLs are included. The API is widely used in digital humanities, journalism studies, and computational media research for building corpora, trend analysis, and topic modeling against news metadata, while full-text mining typically requires separate licensed access through ProQuest or NYT institutional agreements.
Taylor & Francis	Taylor & Francis hosts a very large corpus of scholarly journals and books across the humanities and social sciences as well as STEM fields, with particularly deep coverage in cultural studies, history, education, media studies, and the social sciences. It also includes both contemporary and historical materials and supports programmatic large-scale analysis of full text, which makes it useful for trend analysis, discourse studies, topic modeling, and longitudinal claims about scholarly language or concepts. Because its platform encompasses many influential journals across multiple disciplines.
Springer / BMC	Springer offers a public RESTful API (via the BMC platform) that provides structured access to open-access journal content in JSON or PAM formats, making it easy to integrate into computational workflows in R. The platform also maintains rich article-level metadata and real-time usage and citation metrics, which are especially useful for bibliometrics, scholarly communication research, and longitudinal studies of academic impact.

Source	Description
Library of Congress	The Library of Congress is one of the most valuable resources for large-scale text and data mining because it serves as both a national library and a long-term digital preservation infrastructure with extensive open collections spanning newspapers, manuscripts, maps, recordings, photographs, and government documents. Many of its holdings are digitized and made available through platforms like Congress.gov, Chronicling America, and the LoC Digital Collections, with rich metadata and machine-readable formats that support computational analysis. Its breadth across U.S. history, law, politics, culture, and media — combined with stable hosting and long-term stewardship — makes it a foundational corpus for humanities, social science, and civic-data research, especially for longitudinal or historically grounded projects.
ERIC	ERIC (the Education Resources Information Center) is a major open-access index of research in education, sponsored by the U.S. Department of Education, and it is one of the strongest resources for text and data mining in the learning sciences, education policy, and pedagogy. It offers structured metadata for journal articles, reports, dissertations, curricula, and grey literature from government and nonprofit education organizations, much of it available in full text. Because ERIC aggregates both peer-reviewed research and practitioner-oriented materials, it supports both scholarly analysis and policy-oriented studies, making it ideal for large-scale text analysis of educational trends, terminology shifts, interventions, and discourse over time.
Chronicling America	Chronicling America is a major open-access newspaper portal operated by the Library of Congress in partnership with the National Endowment for the Humanities, and it is one of the most important historical text corpora available for large-scale humanities research. It provides digitized U.S. newspapers spanning 1777–1963 along with rich bibliographic metadata and OCR text, making it well-suited for computational analysis of language, place-based reporting, public discourse, and cultural change over time. Because the platform is built for long-term preservation and public access, it is widely used for projects in digital history, media studies, and historical linguistics, particularly those requiring geographically and temporally broad sources.
Unpaywall	Unpaywall is a powerful resource for large-scale scholarly research because it provides open, programmatic access to metadata about which versions of over 54 million academic articles are freely available on the web. It maintains a massive database of OA copies harvested from repositories, preprint servers, and publisher pages, and its API allows researchers to look up open versions of paywalled articles using DOIs. Rather than hosting the articles itself, Unpaywall acts as a discovery layer for legal open-access versions, which makes it especially useful for building large corpora of scholarly texts without needing direct publisher agreements or subscriptions.

Source	Description
JSTOR	JSTOR is one of the largest digital libraries of scholarly journals, books, and primary sources spanning the humanities and social sciences, with particular strength in older journal runs (deep backfiles) and historically oriented scholarship. For computational text analysis, JSTOR provides Data for Research (DfR), a service that allows researchers to obtain structured metadata, n-grams, and full-text datasets derived from its corpus. It enables bulk export of data in machine-readable formats such as JSON or XML. Because JSTOR's core holdings are largely in-copyright, access to full text is controlled: researchers typically begin with n-gram or metadata-level exports, then may apply for privileged access upon IRB-style justification or institutional agreement. JSTOR is valuable for linguistic, conceptual, and discourse analysis over time, especially in fields like history, literature, cultural studies, anthropology, philosophy, feminist theory, political thought, and other disciplines where journals preserved by JSTOR often represent the canonical scholarly record across the 19th–21st centuries.

RSS Feeds

RSS feeds are a simpler alternative to APIs for data extraction. For example, BBC News offers an RSS feed that returns content in XML format, which can be parsed to extract relevant information.

In the following example, only the titles of the news articles and their associated hyperlinks are retrieved from the feed.[13]

```r
# Load the packages for working with XML and web pages
library(rvest) # Read and scrape web pages
library(xml2) # Read and parse XML files

# Read the BBC News RSS feed
# - read_xml() downloads and parses the XML document
rss_url <- "https://feeds.bbci.co.uk/news/rss.xml"

rss_page <- read_xml(rss_url) # Read the RSS feed

# Extract the article titles and links
# - xml_find_all() finds all matching XML nodes
# - xml_text() extracts the text from each node
# - tibble() stores the results in a data frame
articles <- tibble(
  title = rss_page %>%
    xml_find_all("//item/title") %>% # Find all article titles
    xml_text(), # Extract the title text
  link = rss_page %>%
    xml_find_all("//item/link") %>% # Find all article links
    xml_text()) # Extract the link text

print(head(articles, 5))
```

```
## # A tibble: 5 x 2
##   title                                                                link
##   <chr>                                                                <chr>
## 1 US launches 'heavy' strikes on Iran after attempted attack on American ~ http~
## 2 Thousands evacuated in Crete as Europe battles wildfires             http~
## 3 Large rise in offenders sent back to prison, figures reveal          http~
## 4 Watch: Giant iceberg turns over near  Greenland's coast              http~
## 5 Olivia Dean nominated and  Lily Allen snubbed in Mercury Prize list  http~
```

Bulk Downloads

Bulk downloads from institutional repositories are a common alternative to collecting data through a web service like an API. When working with large datasets, analysts may find bulk download options-—when available-—to be more efficient than relying solely on an API, which can take time to collect the data.

While a bulk download option might seem straightforward, it is often overlooked because inconsistencies in how repositories are structured or labeled can make bulk download options difficult to locate.

Making use of bulk downloads requires attention to file formats, file size, and licensing restrictions. Data may be packaged in tabular formats like CSV or TSV, which mirror the data frames we have been processing throughout this book. Other common formats include JSON and XML, which store data in hierarchical, nested structures often used for representing complex relationships or metadata.

Bulk downloads also include particular challenges for the data analyst. Bulk downloads often include the entire dataset in a single file. Opening such a large file can place significant demands on one's computer resources. In some cases, the file size may exceed the available memory, making it impossible to load the dataset into R (or another coding environment) without additional preprocessing or segmentation. We engage this issue below.

A notable limitation on bulk downloads is that the data is "static" and does not update dynamically, as is typically the case with API-based access. In other words, analysts may not receive the most current or up-to-date information if the dataset is still considered "live", such as is the case with accessing police reports in Dallas. This concern is generally less applicable for historical datasets that are fixed to a past time period, unless records are still being uncovered and added to the collection.

Examples of Bulk Download Websites

A representative example of a bulk download resource is provided by Old Bailey Online, which states:

"If you require the complete data (or the API is not suitable for your needs), it is available in XML format only from the University of Sheffield's data repository (ORDA): http://dx.doi.org/10.15131/shef.data.4775434."[14]

The dataset referenced includes 2,163 editions of the Proceedings and 475 Ordinary's Accounts, all encoded in XML.

Other popular sites for bulk downloads include: * the HathiTrust, a large-scale collaborative repository of digitized texts from academic, historic, and research institutions. Hathitrust offers materials such as historical magazines and newspapers, government documents, and literary works. * Harvard Dataverse, an open-access data repository designed to share, cite, and preserve research data across disciplines. It hosts datasets from a wide range of academic fields, including political science surveys, public health data, economics experiments, and qualitative social science studies. * Kaggle, a data science platform perhaps best known for its competitions, also hosts a wide array of community-contributed datasets, ranging from open science research (such as climate or genomics data) to pop culture topics (like movie ratings, video game sales, or lyrics datasets).[15]

Working With Bulk Download Data

Importantly, while bulk data downloads seem straightforward, they can be difficult to work with in practice. Many open data portals offer complete dataset exports in formats such as CSV or JSON. The primary advantage of accessing a bulk download is the ability to collect substantial volumes of data in a single step.

Size of the download is another concern. A dataset may appear manageable in size because it is compressed–for example, as a .zip file–but unzipping it can produce gigabytes of text that overwhelm the memory or storage capacity of a typical laptop.Even when extraction is successful, trying to load the entire dataset into R with a single command like `read_csv()` or `fromJSON()` can cause the session to crash or become unresponsive.

The `hansardr` package was built with the challenge of loading large data in mind: the full Hansard corpus was broken into smaller partitions to make it possible to load and analyze a large data set without consuming too many resources. But for other datasets, the analyst may need to "chunk" the data manually to make it manageable on a smaller machine.

"Chunking" data is the key approach for an analyst who needs to break a large dataset down into smaller, more workable sections.

Reading in a Large Dataset in Chunks

In cases where large dataset has not been partitioned, it is often necessary to process the data in smaller chunks, such as reading one line at a time or loading a subset of rows using functions like `read_csv_chunked()` from the `tidyverse`'s `readr` or `fread(nrows = ...)` from the `"data.table"()` package.

Below is an example of code for reading a large dataset in 1000 row chunks. For the sake of our practice, we will not download a large dataset onto our computers. We are simply offering a code example.

```
# Load data.table for reading large datasets
library(data.table)

# Read the first 1,000 rows of the dataset
# - fread() imports data from a file
# - nrows specifies how many rows to read
# - skip specifies how many rows to skip before reading
chunk1 <- fread("hansard_large.csv",
                nrows = 1000, # Read the first 1,000 rows
                skip = 0) # Skip no rows

# Read the second 1,000 rows of the dataset
# - fread() imports another chunk of the file
chunk2 <- fread("hansard_large.csv",
                nrows = 1000, # Read 1,000 rows
                skip = 1000) # Skip the first 1,000 rows
```

Below is the same process but automated, so the user will read up to 100,000 rows of the data.

```
# Set parameters for reading data
file_path <- "hansard_large.csv"
chunk_size <- 1000
max_rows <- 100000

# Loop over the file in chunks
```

```r
for (start_row in seq(0, max_rows, by = chunk_size)) {
  chunk <- fread(file_path, nrows = chunk_size, skip = start_row)

  # Do something to the data.
  # For example, the analyst could count the number of words
  # in the Hansard debates, save just the word count, but release the
  # Hansard debates to save memory.
}
```

Downloading and Scraping PDFs

When scholars and libraries work with data, frequently what they make available is not a structured file at all, but a series of PDFs – portable document format files that, once opened, have the appearance of a printed book. PDFs are often favored by scholars who appreciate the graphic format of print.

For the purposes of learning how to read in a pdf to R, let's explore how the process might work using a relatively unproblematic pdf composed of machine-readable text.

Please note that most pdfs that analysts might find on the web are not so well formatted, and they will have formatting or cleaning issues that the analyst must deal with – and which we will turn to in a later section.

Practicing with PDFs: Egil's Saga

It is possible to download a PDF file directly to one's computer and extract its text content for text mining. The following example demonstrates how the process of downloading PDFs can be automated.

We will practice with an 1893 translation of *Egil's Saga,* or the *Skalla-Grímssonar*, one of the classic Icelandic sagas, thought to be written in the 13th century, likely by Snorri Sturluson. We will download *Egil's Saga* from The Icelandic Saga Database, an online resource that has published a large body of medieval Icelandic literature that describes the events that took place among the Norse and Celtic inhabitants of Iceland during the 10th and 11th centuries CE.[16]

Egil's Saga blends historical narrative, legend, and poetry, while focusing on Egill Skallagrímsson, a fierce warrior and gifted poet. The copy of the text that we are using has been previously OCR'd from an original book, rendered as text, cleaned, and output as a machine-readable text with very little noise – making it an ideal subject on which to practice reading in data from a PDF.

Please note: The code that follows handles downloading and text extraction, but it does not include functionality for deleting the file afterward. Our omission is intentional: to prevent any risk of accidentally deleting important or unrelated files, we have left the task of file deletion to the user's discretion. If the script is executed, the reader should manually delete the downloaded PDF file from their computer once it is no longer needed.[17]

```r
# Load the packages for downloading and reading PDF files
library(httr) # Download files from the web
library(fs) # Work with files and directories
library(pdftools) # Extract text from PDF files
```

```
## Using poppler version 22.02.0
```

```r
# Define the PDF to download
url <- "https://sagadb.org/files/pdf/egils_saga.en.pdf"
```

```r
# Define where the PDF will be saved
# - path() creates a file path that works across operating systems
destfile <- path("tmha_data", "egils_saga.en.pdf")

# Create the download directory if it does not already exist
# - path_dir() extracts the directory from the file path
# - dir_create() creates the directory
dir_create(path_dir(destfile))

# Download the PDF
# - GET() downloads the file
# - write_disk() saves it to the specified location
GET(url, write_disk(destfile, overwrite = TRUE))
```

```
## Response [https://sagadb.org/files/pdf/egils_saga.en.pdf]
##   Date: 2026-07-30 13:44
##   Status: 200
##   Content-Type: application/pdf
##   Size: 532 kB
## <ON DISK>  /home/stephbuongiorno/Repos/text-mining-for-historical-analysis/chapter_10_data/tmha_data/
```

```r
# Extract the text from the PDF
# - pdf_text() returns one character string per page
pdf_text_raw <- pdf_text(destfile)

# Create a data frame with one row per page
# - seq_along() generates the page numbers
# - tibble() stores the page numbers and text
tidy_egils_saga <- tibble(page = seq_along(pdf_text_raw), # Number the pages
                          text = pdf_text_raw) # Store the text from each page
```

The result is a data frame (specifically, a tibble) named `tidy_egils_saga`, which has two columns: one column contains the page numbers, and the other contains the corresponding body text from each page.

```r
# Show pages 50 through 60 of the PDF
# - slice() keeps the specified rows
tidy_egils_saga %>%
  slice(50:60)
```

```
## # A tibble: 11 x 2
##     page text
##    <int> <chr>
##  1    50 "                                                  Egil's Saga\n\n\n\nround~
##  2    51 "                                                     Egil's Saga\n\n\n\n  ~
##  3    52 "                                                   Egil's Saga\n\n\n\n   T~
##  4    53 "                                                  Egil's Saga\n\n\n\nhome.~
##  5    54 "                                                  Egil's Saga\n\n\n\n   Bu~
##  6    55 "                                                 Egil's Saga\n\n\n\nChapte~
##  7    56 "                                                  Egil's Saga\n\n\n\nas a ~
##  8    57 "                                                   Egil's Saga\n\n\n\nbefo~
##  9    58 "                                                 Egil's Saga\n\n\n\nbeen g~
## 10    59 "                                                      Egil's Saga\n\n\n\nm~
## 11    60 "                                                   Egil's Saga\n\n\n\nagai~
```

With just a few lines of code to locate, download, and scrape the PDF file, it is now easy to count the top words in *Egil's Saga*.

```
# Load the tidytext stop word list
data("stop_words")

# Find the most common words in Egil's Saga
# - unnest_tokens() splits the text into individual words
# - anti_join() removes common stop words
# - count() counts the occurrences of each word
# - slice_max() keeps the 20 most frequent words
top_words <- tidy_egils_saga %>%
  unnest_tokens(word, text) %>% # Tokenize the text into words
  anti_join(stop_words, by = "word") %>% # Remove common stop words
  count(word, sort = TRUE) %>% # Count the occurrences of each word
  slice_max(n, n = 20) # Keep the 20 most frequent words

# Visualize the most common words
# - aes() maps words and word counts to the plot
# - geom_col() creates a bar chart
# - coord_flip() flips the axes for readability
# - labs() adds the title and axis labels
# - theme_minimal() applies a minimal theme
top_words %>%
  ggplot(aes(x = reorder(word, n), # Order words by frequency
             y = n)) + # Plot word counts
  geom_col() + # Create the bars
  coord_flip() + # Flip the axes
  labs(title = "Top 20 Words in Egil's Saga (after removing stop words)", # Add the chart title
       x = "Word", # Label the x-axis
       y = "Count") + # Label the y-axis
  theme_minimal() # Apply a minimal theme
```

For transparency and book keeping, the analyst may want to add additional metadata—such as the book's title (Egil's Saga), the author (Snorri Sturluson), and the publication date of the English translation (1893).

```
# Add metadata to the Egil's Saga dataset
# - mutate() creates new columns for the author, book title, and publication date
tidy_egils_saga_with_metadata <- tidy_egils_saga %>%
  mutate(author = "Snorri Sturluson", # Record the author
         book = "Egil's Saga", # Record the book title
         date = "1893") # Record the publication date
```

The metadata helps an analyst keep track of key information about the book. However, because each row in the author, book, and date columns contains the same values, the metadata is redundantly stored in the data frame.

```
# Show the first six rows of the dataset
# - head() keeps the first six rows
# - kable() formats the results as a table
tidy_egils_saga_with_metadata %>%
  head() %>% # Keep the first six rows
  kable() # Format the results as a table
```

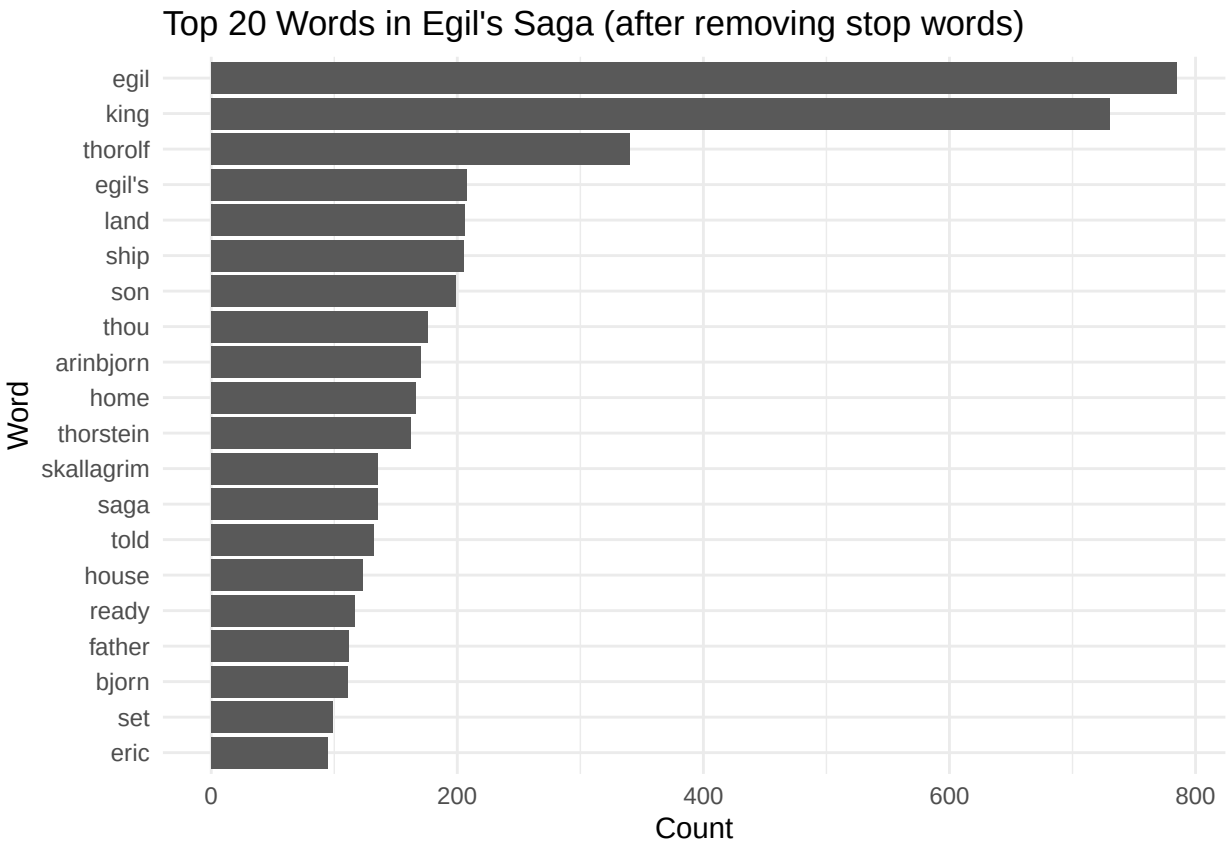


Figure 2: Top 20 words in Egil's Saga with stop words removed.

1 Egil's Saga

1893 translation into English by W. C. Green from the original Icelandic 'Egils saga Skallagrímssonar'.

Chapter 1 - Of Kveldulf and his sons. There was a man named Ulf, son of Bjalf, and Hallbera, daughter of Ulf the fearless; she was sister of Hallbjorn Half-giant in Hrafnista, and he the father of Kettle Hæing. Ulf was a man so tall and strong that none could match him, and in his youth he roved the seas as a freebooter. In fellowship with him was one Kari of Berdla, a man of renown for strength and daring; he was a Berserk. Ulf and he had one common purse, and were the dearest friends. But when they gave up freebooting, Kari went to his estate at Berdla, being a man of great wealth. Three children had Kari, one son named Eyvind Lambi, another Aulvir Hnuf, and a daughter Salbjorg, who was a most beautiful woman of a noble spirit. Her did Ulf take to wife, and then he too went to his estates. Wealthy he was both in lands and chattels; he took baron's rank as his forefathers had done, and became a great man. It was told of Ulf that he was a great householder; it was his wont to rise up early, and then go round among his labourers or where his smiths were, and to overlook his stalk and fields, and at times he would talk with such as needed his counsel, and good counsel he could give in all things, for he was very wise. But everyday as evening drew on he became sullen, so that few could come to speak with him. He was an evening sleeper, and it was commonly said that he was very shape strong. He was called Kveldulf. Kveldulf and his wife had two sons, the elder was named Thorolf, the younger Grim; these, when they grew up, were both tall men and strong, as was their father. But Thorolf was most comely as well as doughty, favoring his mother's kin; very cheery was he, liberal, impetuous in everything, a good trader, winning the hearts of all men. Grim was swarthy, ill-favoured, like his father both in face and mind; he became a good man of business; skilful was he in wood and iron, an excellent smith. In the winter he often went to the herring fishing, and with him many house-carles. But when Thorolf was twenty years old, then he made him ready to go a harrying. Kveldulf gave him a long-ship, and Kari of Berdla's sons, Eyvind and Aulvir, resolved to go on that voyage, taking a large force and another long-ship; and they roved the seas in the summer, and got them wealth, and had a large booty to divide. For several summers they were out roving, but stayed at home in winter with their fathers. Thorolf brought home many costly things, and took them to his father and mother; thus they were well-to-do both for possessions and honour. Kveldulf was now well stricken in years, and his sons were grown men.

Chapter 2 - Of Aulvir Hnuf. Audbjorn was then king over the Firthfolk; there was an earl of his named Hroald, whose son was Thorir. Atli the Slim was then an earl, he dwelt at Gaula; he had sons - Hallstein, Holmstein, and Herstein; and a daughter, Solveig the Fair. It happened one autumn that much people were gathered at Gaula for a sacrificial feast, then saw Aulvir Hnuf Solveig and courted her; he afterwards asked her to wife. But the earl thought him an unequal match and would not give her. Whereupon Aulvir composed many love-songs, and thought so much of Solveig that he left freebooting, but Thorolf and Eyvind Lambi kept it on.

Chapter 3 - The beginning of the rule of Harold Fairhair. Harold, son of Halfdan Swarthy, was heir after his father. He had bound himself by this vow, not to let his hair be cut or combed till he were sole king over Norway, wherefore he was called Harold Shockhead. So first he warred with the kings nearest to him and conquered them, as is told at length elsewhere. Then he got possession of Upland; thence he went northwards to Throndheim, and had many battles there before he became absolute over all the Thronds. After that he purposed to go north to Naumdale to attack the brothers Herlaug and Hrollaug, kings of Naumdale. But when these brothers heard of his coming, Herlaug with twelve men entered the sepulchral mound which they had caused to be made (they were three winters at the making), and the mound then was closed after them.

But king Hrollaug sank from royalty to earldom, giving up his kingdom and becoming a vassal of Harold. So Harold gained the Naumdalesmen and Halogaland, and he set rulers over his realm there. Then went he southwards with a fleet to Mæra and Raumsdale. But Solvi Bandy-legs, Hunthiof's son, escaped thence, and going to king Arnvid, in South Mæra, he asked help, with these words: 'Though this danger now touches us, before long the same will come to you; for Harold, as I ween, will hasten hither when he has enthralled and oppressed after his will all in North Mæra and Raumsdale. Then will the same need be upon you as was upon us, to guard your wealth and liberty, and to try everyone from whom you may hope for aid. And I now offer myself with my forces against this tyranny and wrong. But, if you make the other choice, you must do as the Naumdalesmen have done, and go of your own will into slavery, and become Harold's thralls. My father though it victory to die a king with honour rather than become in his old age another king's subject. Thou, as I judge, wilt think the same, and so will others who have any high spirit and claim to be men of valour.' By such persuasion king Arnvid was determined to gather his forces and defend his land. He and Solvi made a league, and sent messengers to Audbjorn, king of the Firthfolk, that he should come and help them. Audbjorn, after counsel taken with friends, consented, and bade cut the war-arrow and send the war-summons throughout his realm, with word to his nobles that they should join him. But when the king's messengers came to Kveldulf and told him their errand, and that the king would have Kveldulf come to him with all his house-carles, then answered he: 'It is my duty to the king to take the field with him if he have to defend his own land, and there be harrying against the Firthfolk; but this I deem clean beyond my duty, to go north to Mæra and defend their land. Briefly ye may say when ye meet your king that Kveldulf will sit at

home during this rush to war, nor will he gather forces nor leave his home to fight with Harold Shockhead. For I think that he has a whole load of good-fortune where our king has not a handful.' The messengers went back to the king, and told him how their errand had sped; but Kveldulf sat at home on his estates.

Chapter 4 - Battle of king Harold and Audbjorn. King Audbjorn went with his forces northwards to Mæra; there he joined king Arnvid and Solvi Bandy-legs, and altogether they had a large host. King Harold also had come from the north with his forces, and the armies met inside Solskel. There was fought a great battle, with much slaughter in either host. Of the Mærian forces fell the kings Arnvid and Audbjorn, but Solvi escaped, and afterwards became a great sea-rover, and wrought much scathe on Harold's kingdom, and was nicknamed Bandy-legs. On Harold's side fell two earls, Asgaut and Asbjorn, and two sons of earl Hacon, Grjotgard and Herlaug, and many other great men. After this Harold subdued South Mæra. Vemund Audbjorn's brother still retained the Firthfolk, being made king. It was now autumn, and king Harold was advised not to go south in autumn-tide. So he set earl Rognvald over North and South Mæra and Raumsdale, and kept a numerous force about himself. That same autumn the sons of Atli set on Aulvir Hnuf at his home, and would fain have slain him. They had such a force that Aulvir could not withstand them, but fled for his life. Going northwards to Mæra, he there found Harold, and submitted to him, and went north with the king to Throndheim, and he became most friendly with him, and remained with him for a long time thereafter, and was made a skald. In the winter following earl Rognvald went the inner way by the Eid-sea southwards to the Firths. Having news by spies of the movements of king Vemund, he came by night to Naust-dale, where Vemund was at a banquet, and, surrounding the house, burnt within it the king and ninety men. After that Karl of Berdla came to earl Rognvald with a long-ship fully manned, and they two went north to Mæra. Rognvald took the ships that had belonged to Vemund and all the chattels he could get. Kari of Berdla then went north to king Harold at Throndheim, and became his man. Next spring king Harold went southwards along the coast with a fleet, and subdued firths and fells, and arranged for men of his own to rule them. Earl Hroald he set over the Firthfolk. King Harold was very careful, when he had gotten new peoples under his power, about barons and rich landowners, and all those whom he suspected of being at all likely to raise rebellion. Every such man he treated in one of two ways: he either made him become his liege-man, or go abroad; or (as a third choice) suffer yet harder conditions, some even losing life or limb. Harold claimed as his own through every district all patrimonies, and all land tilled or untilled, likewise all seas and freshwater lakes. All landowners were to be his tenants, as also all that worked in the forest, salt-burners, hunters and

fishers by land and sea, all these owed him duty. But many fled abroad from this tyranny, and much waste land was then colonized far and wide, both eastwards in Jamtaland and Helsingjaland, and also the West lands, the Southern isles, Dublin in Ireland, Caithness in Scotland, and Shetland. And in that time Iceland was found.

3

4|Egil's Saga

Chapter 5 - The king's message to Kveldulf. King Harold lay with his fleet in the Firths, whence he sent messengers round the land to such as had not come to him, but with whom he thought he had business. The messengers came to Kveldulf, and were well received. They set forth their errand, said that the king would have Kveldulf come to him. 'He has heard,' said they, 'that you are a man of renown and high family. You will get from him terms of great honour, for the king is very keen on this, to have with him such as he hears are men of mark for strength and bravery.' Kveldulf answered that he was an old man, not fit for war or to be out in warships. 'I will now,' said he, 'sit at home and leave serving kings.' Upon this the messengers said, 'Then let your son go to the king; he is a tall man and a likely warrior. The king will make you a baron,' said they to Grim, 'if you will serve him.' 'I will be made baron under none,' said Grim, 'while my father lives; he, while he lives, shall be my liege-lord.' The messengers went away, and when they came to the king told him all that Kveldulf had said before them. Whereat the king looked sullen, but he spoke little; these men, he said, were proud, or what were they aiming at? Aulvir Hnuf was standing near, and he bade the king not be wroth. 'I will go,' said he, 'to Kveldulf; and he will consent to come to you, as soon as he knows that you think it a matter of moment.' So Aulvir went to Kveldulf and told him that the king was wroth, and it would not go well unless one of the two, father or son, came to the king; he said, too, that he would get them great honour from the king if they would but pay homage. Further he told them at length, as was true, that the king was liberal to his men both in money and in honours. Kveldulf said, 'My foreboding is that I and my sons shall get no luck from this king: and I will not go to him. But if Thorolf returns this summer, he will be easily won to this journey, as also to be made the king's man. Say you this to the king, that I will be his friend, and will keep to his friendship all who heed my words; I will also hold the same rule and authority from his hand that I held before from the former king, if he will that it continue so still, and I will see how I and the king agree.' Then Aulvir went back and told the king that Kveldulf would send him his son, and he (said Aulvir) would suit better; but he was not then at home. The king let the matter rest. In the summer he went inland to Sogn, but in autumn made ready to go northwards to Throndheim.

Chapter 6 - Thorolf resolves to serve the king. Thorolf Kveldulf's son and Eyvind Lambi came home from sea-roving in the autumn. Thorolf went to his father, and father and son had some talk together. Thorolf asked what had been the errand of the men whom Harold sent thither. Kveldulf said the king had sent them with this message, that Kveldulf or else one of his sons should become his man. 'How answeredst thou?' said Thorolf. 'I spake what was in my mind, that I would never take service with king Harold; and ye two will both do the same, if I may counsel: this I think will be the end, that we shall reap ruin from that king.'

4

5|Egil's Saga

'That,' said Thorolf, 'is quite contrary to what my mind tells me, for I think I shall get from him much advancement. And on this I am resolved, to seek the king, and become his man; and this I have learnt for true, that his guard is made up of none but valiant men. To join their company, if they will have me, seems to me most desirable; these men are in far better case than all others in the land. And 'tis told me of the king that he is most generous in money gifts to his men, and not slow to give them promotion and to grant rule to such as he deems meet for it. Whereas I hear this about all that turn their backs upon him and pay him not homage with friendship, that they all become men of nought, some flee abroad, some are made hirelings. It seems wonderful to me, father, in a man so wise and ambitious as thou art, that thou wouldst not thankfully take the dignity which the king offered thee. But if thou thinkest that thou hast prophetic foresight of this, that we shall get misfortune from this king, and that he will be our enemy,

then why didst thou not go to battle against him with that king in whose service thou wert before? Now, methinks it is most unreasonable neither to be his friend nor his enemy.' 'It went,' said Kveldulf, 'just as my mind foreboded, that they marched not to victory who went northwards to fight with Harold Shockhead in Mæra; and equally true will this be, that Harold will work much scathe on my kin. But thou, Thorolf, wilt take thine own counsel in thine own business; nor do I fear, though thou enter into the company of Harold's guards, that thou wilt not be thought capable and equal to the foremost in all proofs of manhood. Only beware of this, keep within bounds, nor rival thy betters; thou wilt not, I am sure, yield to others overmuch.' But when Thorolf made him ready to go, Kveldulf accompanied him down to the ship and embraced him, with wishes for his happy journey and their next merry meeting.

Chapter 7 - Of Bjorgolf, Brynjolf, Bard, and Hildirida. There was a man in Halogaland named Bjorgolf; he dwelt in Torgar. He was a baron, powerful and wealthy; in strength, stature, and kindred half hill-giant. He had a son named Brynjolf, who was like his father. Bjorgolf was now old, and his wife was dead; and he had given over into his son's hands all business, and found him a wife, Helga, daughter of Kettle Hæing of Hrafnista. Their son was named Bard; he soon grew to be tall and handsome, and became a right doughty man. One autumn there was a banquet where many men were gathered, Bjorgolf and his son being there the most honourable guests. In the evening they were paired off by lot to drink together, as the old custom was. Now, there was at the banquet a man named Hogni, owner of a farm in Leka, a man of great wealth, very handsome, shrewd, but of low family, who had made his own way. He had a most beautiful daughter, Hildirida by name; and it fell to her lot to sit by Bjorgolf. They talked much together that evening, and the fair maiden charmed the old man. Shortly afterwards the banquet broke up. That same autumn old Bjorgolf journeyed from home in a cutter of his own, with thirty men aboard. He came to Leka, and twenty of them went up to the house, while ten guarded the ship. When they came to the farm, Hogni went out to meet him, and made him welcome, invited him and his comrades to lodge there, which offer Bjorgolf accepted, and they entered the room. But when they had doffed their travelling clothes and donned mantles, then Hogni gave orders to bring in a large bowl of beer; and Hildirida, the daughter of the house, bare ale to the guests.

```
5 |Snorri Sturluson |Egil's Saga |1893 |
```

6|Egil's Saga

Bjorgolf called to him Hogni the goodman, and said, 'My errand here is this: I will have your daughter to go home with me, and will even now make with her a hasty wedding.' Hogni saw no choice but to let all be as Bjorgolf would; so Bjorgolf bought her with an ounce of gold, and they became man and wife, and Hildirida went home with Bjorgolf to Torgar. Brynjolf showed him ill-pleased at this business. Bjorgolf and Hildirida had two sons; one was named Harek, the other Hærek. Soon after this Bjorgolf died; but no sooner was he buried than Brynjolf sent away Hildirida and her sons. She went to her father at Leka, and there her sons were brought up. They were good-looking, small of stature, naturally shrewd, like their mother's kin. They were commonly called Hildirida's sons. Brynjolf made little count of them, and did not let them inherit aught of their father's. Hildirida was Hogni's heiress, and she and her sons inherited from him and dwelt in Leka, and had plenty of wealth. Bard, Brynjolf's son, and Hildirida's sons were about of an age. Bjorgolf and his son Brynjolf had long held the office of going to the Finns, and collecting the Finns' tribute. Northwards, in Halogaland is a firth called Vefsnir, and in the firth lies an island called Alost, a large island and a good, and in this a farm called Sandness. There dwelt a man named Sigurd, the richest man thereabouts in the north; he was a baron, and wise of understanding. He had a daughter named Sigridr; she was thought the best match in Halogaland, being his only child and sole heiress to her father. Bard Brynjolf's son journeyed from home with a cutter and thirty men aboard northwards to Alost, and came to Sigurd at Sandness. There he declared his business, and asked Sigridr to wife. This offer was well received and favourable answered, and so it came about that Bard was betrothed to the maiden. The marriage was to take place the next summer. Bard was then to come north for the wedding.

Chapter 8 - Of Bard and Thorolf. King Harold had that summer sent word to the men of power that were in Halogaland, summoning to him such as had not come to him before. Brynjolf resolved to go, and with him Bard his son; and in the autumn they went southwards to Throndheim, and there met the king. He

received them most gladly. Brynjolf was made a baron of the king's; the king also gave him large grants beside what he had before. He gave him withal the right of journey to the Finns, with the king's business on the fells and the Finn traffic. Then Brynjolf went away home to his estate, but Bard remained, and was made one of the king's guard. Of all his guard the king most prized his skalds; they occupied the second high seat. Of these Audun Ill-skald sat innermost, being the oldest; he had been skald to Halfdan Swarthy, king Harold's father. Next to him sat Thorbjorn Raven, then Aulvir Hnuf, and next to him was placed Bard; he was there by-named Bard the White or Bard the Strong. He was in honour with everyone there, but between him and Aulvir Hnuf was a close friendship. That same autumn came to king Harold Thorolf Kveldulf's son and Eyvind Lambi, Kari of Berdla's son, and they were well received. They brought thither a swift twenty-benched long-ship well manned, which they had before used in sea-roving. They and their company were placed in the guest-hall; but when they had waited there till they thought it a fit time to go before the king, Kari of Berdla and Aulvir Hnuf went in with them. They greeted the king. Then said Aulvir Hnuf,

6

Getting PDFs Ready for Computer Readability: Optical Character Recognition (OCR)

The foregoing exmaple used a deliberately simple-to-read PDF to demonstrate a process. However, in real life, the PDFs collected by analysts or digitalized by museums, archives, corporations, or government institutions may or may not have been prepared for computer readability.

One rule of thumb for deciding whether a PDF is machine readable is to open the document in a computer application like Adobe or Preview, designed for examining PDFs. If the text on the page of the PDF can be highlighted by the user of an app, then the text is ready for digital processing, i.e. it has already been "OCR'd," or provided to an algorithm called "optical character recognition," which uses visual pattern detection to recognize the text on the page.

In the past, analysts typically paid for proprietary software when they wanted to apply OCR to PDF files, and the OCR was of varying quality, with OCR packages from twenty years ago producing many errors, and more recent OCR showing a marked improvement and occasionally the ability to identify handwritten scripts from the past, especially when the OCR program had been trained on many examples of the script translated by patient scholars.

The results are promising. For printed Ottoman Turkish, scholars using platforms like Transkribus have trained Handwritten Text Recognition (HTR) models on ~386 pages of late-Ottoman periodicals, achieving a character error rate of ~7.2 % as of June 2023.[18] Printed Devanagari (Hindi, Sanskrit, etc.) has been widely supported through engines like Tesseract, E-Aksharayan, and customized models (e.g. C-DAC). Chinese handwritten character recognition is a long-standing research domain. Techniques such as DenseRAN (radical-aware networks) and recurrent neural net architectures (LSTM/GRU) have achieved high accuracy in offline recognition contexts—even on large character sets up to 30,000 characters, as shown in industry systems (e.g. Apple's Scribble) and academic datasets like ICDAR-2013 with strong performance.[19]

Today, OCR can be performed using AI tools, typically at a fraction of the cost. Although best practices in text recognition are beyond the scope of this book, we have heard reports of scholars having success with using AI to recognize hand-written text in Latin and Spanish using out-of-the-box LLMs such as ChatGPT.

A cursory review of recent articles suggests that the best results for handwritten records involve extensive training sets, translation, and sampling. Breakthrough success has been reported for using AI trained on Chinese characters to recognize Chinese texts.[20] A Stanford team is applying deep ML and custom training to recognize handwritten text from records in Ottoman, Classical Arabic, and Ottoman Turkish.[21] We read of good results on Ancient Indian records from Brahmi, Grantha, Devanagari and Tamil manuscripts, ancient Chinese records, handwritten East Syriac, and Latin registers from the Vatican.[22] The reports suggest that text mining for historical analysis will soon be applied to a widening collection of scholarly texts from different times and places.

Once a collection of PDFs has been OCR'd, it is possible to use the resulting PDF files to create a data frame for text mining.

Web Scraping

Before the widespread availability of APIs, data collection from websites was commonly performed through web scraping, which involves the extraction of information from the underlying HTML structure of web pages. In many cases, web scraping is no longer necessary. Numerous websites now provide structured data access through APIs. But in certain circumstances, web scraping can still help the analyst to prepare information from websites as a dataset.

In R, the `rvest` package facilitates web scraping by parsing HTML content into a structured representation, commonly referred to as the HTML tree.[23]

An additional precaution is to add code to slow down the web scraping process. To avoid overloading the server hosting the webpage, requests are typically rate-limited during web scraping.

Practicing Web Scraping: Retrieving the Code for *Text Mining for Historical Analysis*

The following example illustrates the initial step in web scrapings: retrieving the HTML content of the GitHub repository page for this book followed by a snippet of the page's HTML.

```
# Define the web page to download
url <- "https://github.com/stephbuon/text-mining-for-historical-analysis"

# Read the web page
# - read_html() downloads and parses the HTML document
page <- read_html(url) # Read the HTML page
```

```
## <!DOCTYPE html>
## <html lang="en" data-color-mode="auto" data-light-theme="light" data-dark-theme="dark" data-a11y-...
## <head>
## <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
## <meta charset="utf-8">
```

Understanding HTML Code and Finding the Content

To make practical use of webscraping, the analyst first needs to become aware of the components of the HTML code that structures a website. On a webpage, each element of HTML code gives the computer specific information about what to display, for instance headers, paragraphs, and links. This information is given in a specific structure in the HTML document, which has a tree-like branching structure.

There are two main features of HTML code: * CSS selectors, derived from web design conventions, identify elements based on tags (e.g., "`p`" for paragraphs), classes (e.g., "`.title`"), IDs (e.g., "`#main`"), or structural relationships (e.g., "`div > p`" for paragraph tags directly nested in a `div`). * In contrast, XPath offers a more expressive syntax for specifying paths through the HTML tree, enabling selection based on element attributes, positions, or nested hierarchies (e.g., "`//div[@class='content']/p`" selects all paragraph tags within div elements that have the class content).

As an example, in the following code the CSS selector `p` would select both paragraph elements inside the `<div>`.

```
<div class="article">
  <h2>A Heading</h2>

  <div class="text-block">
    <p>First paragraph.</p>
    <p>Second paragraph.</p>
  </div>

  <ul class="links">
    <li><a href="#">Link one</a></li>
    <li><a href="#">Link two</a></li>
  </ul>
</div>
```

To extract the content itself, the analyst uses selectors to locate relevant elements in the HTML tree and then retrieve their text. In practice, scraping involves three major steps: downloading the page's HTML, parsing it into a structured document, and applying CSS or XPath queries to extract the desired nodes. For example, using the `rvest` package in R, the following code retrieves all paragraph text from a webpage using the `p` tag as the selector:

```r
# Read the web page
# - read_html() downloads and parses the HTML document
url <- "https://example.com"
page <- read_html(url)

# Extract the paragraph text from the web page
# - html_elements() finds all HTML elements matching a CSS selector
# - html_text() extracts the text from those elements
paragraphs <- page %>%
  html_elements("p") %>% # Find all paragraph elements
  html_text(trim = TRUE) # Extract the paragraph text

print(paragraphs)
```

```
## [1] "This domain is for use in documentation examples without needing permission. Avoid use in opera
## [2] "Learn more"
```

Data Cleaning

Historic documents pose unique challenges for text mining due to inconsistencies in spelling and the physical degradation of materials over time. Additionally, digitization often introduces errors, such as mistakes from Optical Character Recognition (OCR), which complicate analysis.

As analysts aim to uncover language patterns and insights within historical texts, data cleaning becomes an important step in preparing documents for analysis. Data cleaning is the process of identifying and correcting errors, inconsistencies, and inaccuracies in data to ensure its quality and reliability.

This section provides analysts with the conceptual foundation and practical tools necessary for effective data cleaning. We argue that data cleaning should not be treated as a purely technical task, but rather as an interpretive and methodological decision: analysts must consider whether data should be cleaned for a given analysis, and if so, determine which forms of cleaning are appropriate in light of the research goals. Our aim is to demonstrate the techniques for cleaning data while remaining critical and aware of the editorial decisions involved.

Identifying Messy Data

Defining "messy" is not a straightforward task. Rather, determining what counts as messy is subjective and should be guided by a clear rationale tied to the goals of a given analysis. Throughout *Text Mining for Historical Analysis*, we have made baseline assumptions about what constitutes "messy" in ways we believe support certain forms of computational inquiry. For example, we provide a version of Hansard–via `hansardr`–that has disambiguated speaker names. This column is meant to address inconsistencies and ambiguities in the original records, as well as remove special symbols or other artifacts that may interfere with our research goals, like debate text appended to a speakers name.[24]

We offer the "cleaned" speaker data because we believe it can help structure the data in a way that allows speech to be attributed to a single member of Parliament. Clean speaker names may be particularly useful for analyses that rely on mapping discourse to individual actors. Similarly, we have removed stop words and eliminated symbols such as brackets or asterisks when they appear in discourse. Through cleaning, we have made certain assumptions about which textual features are considered meaningful for our purposes and which can be treated as noise–assumptions that reflect specific analytical priorities and inevitably shape the kinds of historical arguments that can be made.

Our approach to identifying and managing "messy" data is not shared unilaterally across all domains. Certainly, for some researchers the "mess" we have identified might be considered a rich feature of the historical record that should be considered during an analysis. For instance, Ryan Cordell, has argued that messy data is not a problem to be solved so much as a condition to preserved and engaged critically. He notes that digital artifacts are often treated as though they are meant to serve as faithful replicas of the original texts, as if they are most valuable if they contain the exact words without any OCR errors. Yet, as he argues, when "we treat the digitized object primarily as a surrogate for its analog original, we jettison the most compelling qualities of both media. The unique use of the digital medium, broadly considered, is the capacity to computationally trace patterns across corpora of various sizes."[25] Matthew Kirschenbaum calls for providing an "account of electronic texts as artifacts—mechanisms—subject to material and historical forms of understanding."[26] Kirschenbaum challenges the idea that digital texts are immaterial or purely abstract representations. Instead, when we understand electronic texts as artifacts—objects that are produced, stored, and transmitted through specific material mechanisms (like hard drives, servers, or software systems) we can critically engage with the artifact's existence in the world.

We do not disagree with these points. Texts–especially historical ones–are mediated through layers of transmission, digitization, and encoding. Depending upon one's approach to analysis, to clean too aggressively is to erase and to normalize to heavily is to flatten. Indeed, a historian's job is not just to extract patterns but to understand data provenance. Here, however, we have defined "messy" in a way that we think is meaningful for computational historical analysis, which engages data through text mining. For our purposes here, common signs of messy data include:

- Special characters (e.g. an asterisk in a speaker's name)
- Duplicate rows
- Misspellings
- Numeric values stored as text/character data types
- White space
- Missing values
- Illogical or unintended text concatenations resulting from formatting

From the Hansard data, we provide the following example of an illogical text concatenation: the word "said" is included as part of the speaker's name. As a result, "The Bishop of Exeter" and "The Bishop of Exeter said" appear as two separate speakers in the dataset. If we were to group the data by speaker and count the most frequent words, this error would lead to treating a single individual as two distinct parliamentarians.

```r
# Load the 1840 speaker metadata
# - data() loads the dataset into the R session
data("speaker_metadata_1840")
```

```r
# Display the speaker metadata for a specific sentence
# - filter() keeps the row matching the specified sentence ID
speaker_metadata_1840 %>%
  filter(sentence_id == "S3V0051P0_728") # Keep the matching sentence
```

```
## # A tibble: 1 x 3
##   sentence_id   speaker                   suggested_speaker
##   <chr>         <chr>                     <chr>
## 1 S3V0051P0_728 The Bishop of Exeter said <NA>
```

In the following examples, an apostrophe is included at the beginning of the name "Mr. Speaker" and a hyphen appears before the name "Lord Ashley."

```r
# Do the same for another sentence
speaker_metadata_1840 %>%
  filter(sentence_id == "S3V0060P0_2194")
```

```
## # A tibble: 1 x 3
##   sentence_id    speaker      suggested_speaker
##   <chr>          <chr>        <chr>
## 1 S3V0060P0_2194 'Mr. Speaker charles_shaw-lefevre_3007
```

```r
# Do the same for another sentence
speaker_metadata_1840 %>%
  filter(sentence_id == "S3V0051P0_18193")
```

```
## # A tibble: 1 x 3
##   sentence_id     speaker      suggested_speaker
##   <chr>           <chr>        <chr>
## 1 S3V0051P0_18193 -Lord Ashley anthony_ashley-cooper_2518
```

These are just a few examples from the many typographical anomalies existing within the digitized version of the Hansard corpus.

Data Cleaning Using a Large Langauge Model

The problem of cleaning data is a long one. A common approach to addressing the unwanted symbols, formatting issues, and inconsistencies is through the use of regular expressions (often shortened to "regex"). We have already demonstrated the regular expressions can be used to target specific, unwanted data. Here we briefly demonstrate a very simple use of a regular expression to clean the "'" from the name "Mr. Speaker."

```r
# Display the speaker metadata for a specific sentence
# - filter() keeps the row matching the specified sentence ID
speaker_metadata_1840 %>%
  filter(sentence_id == "S3V0060P0_2194") # Keep the matching sentence
```

```
## # A tibble: 1 x 3
##   sentence_id    speaker      suggested_speaker
##   <chr>          <chr>        <chr>
## 1 S3V0060P0_2194 'Mr. Speaker charles_shaw-lefevre_3007
```

```
# Display the speaker metadata for a specific sentence
# - filter() keeps the row matching the specified sentence ID
# - mutate() removes apostrophes from the speaker name
speaker_metadata_1840 %>%
  filter(sentence_id == "S3V0060P0_2194") %>% # Keep the matching sentence
  mutate(speaker = str_replace_all(speaker, "'", "")) # Remove apostrophes from the speaker name
```

```
## # A tibble: 1 x 3
##   sentence_id    speaker     suggested_speaker
##   <chr>          <chr>       <chr>
## 1 S3V0060P0_2194 Mr. Speaker charles_shaw-lefevre_3007
```

Many guides exist for defining and using regular expressions. Therefore, we will not include one here and instead suggest reading CRAN's vignette on regular expressions (https://cran.r-project.org/web/packages/stringr/vignettes/regular-expressions.html) for a comprehensive encyclopedia of regular expressions in R.

While effective, writing our own regular expression-based approaches to data cleaning can become increasingly intricate and difficult to scale, such as demonstrated by the following code that relies on `stringr` for handling strings.[27]

```
# Messy string
messy_string <- paste0("<p>We!!!   can clean   messy--",
                       "<b> text using </b>.. ,    regular expressions </p>")

# Clean the text
# - str_remove_all() removes HTML tags
# - str_replace_all() removes punctuation
# - the second str_replace_all() replaces multiple spaces with a single space
# - str_trim() removes leading and trailing whitespace
clean_string <- messy_string |>
  str_remove_all("<[^>]+>") |> # Remove HTML tags
  str_replace_all("[^\\w\\s]", "") |> # Remove punctuation
  str_replace_all("\\s+", " ") |> # Replace multiple spaces with a single space
  str_trim() # Remove leading and trailing whitespace

print(clean_string)
```

```
## [1] "We can clean messy text using regular expressions"
```

Writing complex regular expressions can be time-consuming and difficult to manage, making alternative approaches increasingly appealing. Recently, large language models (LLMs) have been explored as tools for data cleaning. Unlike traditional programming methods, LLMs are not bound by rigid symbolic logic. They can instead interpret natural language instructions. Therefore, we can simply describe how we want a messy string cleaned, and the model can perform the task without requiring detailed code.

However, relying on LLMs to directly clean text introduces certain risks. Unlike traditional programmatic methods, which transform text by manipulating the original content, LLMs generate entirely new text based on probabilistic models. Because the output often closely resembles the input, it is easy to mistake this

generated text for a faithful reproduction—when in fact it is newly generated text, not a transference of the original text. Generation makes LLMs more prone to introducing errors or "hallucinations"–that is, producing text that is semantically plausible but factually incorrect. Such risks are heightened when (a) the input text is significantly different from the data the model was trained on–as is often the case with historical or domain-specific texts–or (b) when the model is given an overly large or complex input to process at once.

We have found it most effective to use the LLM to identify issues in the text and generate code—such as R scripts or regular expressions—that can then be executed programmatically to clean the data. Using a LLM to generate code lets us have our cake and eat it too: we benefit from the LLM'ss ability to quickly construct complex cleaning logic, while maintaining the safety, transparency, and reproducibility of programmatic data processing by generating the cleaning code and not perform the cleaning itself. Rather than asking the LLM to rewrite the passage directly and copying its regenerated output, we can preserve the original text while applying transparent, repeatable transformations.

For example, we provided the following prompt and string to OpenAI's GPT-4:

> Write regular expression(s) to clean this text. Return your response as R code. "
>
> We!!! can clean messy– text using .. , regular expressions
>
> "

GPT-4 returned the following code:

```r
library("stringr")

# Remove HTML tags
messy_string <- gsub("<[^>]+>", "", messy_string)

# Replace multiple punctuation marks with a space
# (put hyphen first or last to avoid an error where the code
# goes out of range
messy_string <- gsub("[-!.,]+", " ", messy_string)

# Collapse multiple spaces and trim
text <- str_squish(messy_string)

print(text)
```

```
## [1] "We can clean messy text using regular expressions"
```

Since a single regular expression is often insufficient to handle all types of messy input, it may be desirable to apply this cleaning approach iteratively, for example, by using a `for` loop that calls the API for a LLM and dynamically generates regular expressions tailored to the specific cleaning needs of the given text.

Below, we provide an example of what the iterative steps might look like in pseudo-code, or code that resembles real code in order to communicate an idea, but is not actually intended to run. To use similar code, the reader will need to obtain and configure an API key for access to a LLM.

```
# Repeatedly clean the text using LLM-generated regular expressions
# - the outer for() loop repeats the cleaning process
# - the inner for() loop processes one row at a time
# - call_llm_api() requests a regular expression from the language model
# - str_replace_all() applies the suggested regular expression
max_iterations <- 5
```

```
for (i in 1:max_iterations) {
  for (row in 1:nrow(df)) {

    text <- df$text[row] # Extract the current text

    llm_suggested_regex <- call_llm_api(
      prompt = paste("Suggest a regular expression to further clean this text:\n", text)) # Request a re

    text <- str_replace_all(text, llm_suggested_regex, " ") # Apply the regular expression

    # Update the data frame with the cleaned text
    df$text[row] <- text } } # Save the cleaned text back to the data frame
```

Interpreting Data Format, Type, and Structure

Digital methods are now deeply embedded in many areas of humanities research. As a result, it is increasingly necessary to move beyond a surface-level familiarity with the technical aspects of computational text analysis. We argue it is necessary to approach digital media not merely as "tools," but as complex systems shaped by specific technical features. A conceptual understanding of how data is structured, processed, and interpreted, we suggest, is helpful for engaging more critically with digital sources, and for integrating these methods into the interpretive practices of the humanities.

This section introduces core technical concepts—data format, type, and structure—that shape how textual data is processed and analyzed. In many computational disciplines, an understanding of data type, format, and structure are treated as foundational and are typically addressed early, as they serve as the building blocks to more advanced forms of analysis. Throughout *Text Mining for Historical Analysis* however, we intentionally set these concepts aside because our approach to text mining in R largely handles data complexities. Most of our work involved character data (e.g., words in a debate) and integer data (e.g., frequency counts of top words), which were stored in data frames compatible with the (primarily) `tidyverse` tools we selected. Our deliberate approach allowed us to move straight into historical analysis without worrying about data complexities. Here, we pause to examine the underlying structures more deliberately, as a deeper understanding of them can inform strategies for different types of analyses. This section thus reframes some technical fluency as a critical component of interpretive practice in computational text analysis.

Data Format

At this point in the chapter, we have already examined and engaged with several different data formats. Instead of reiterating the same exercises here, we will provide a summation of data formats.

Data formats refer to the way information is stored and represented. It has an impact on how it is accessed by both analysts and machines, and distinct affordances and trade-offs, depending on the structure and goals of historical analysis.

- CSV/TSV (Comma- or Tab-Separated Values) – Both CSV and TSV are flat tabular formats in which each line represents a row and values are separated by a delimiter–commas for CSV, tabs for TSV. Both formats are well-suited for structured, spreadsheet-like data, but they are less effective for representing complex or hierarchical relationships.
- JSON (JavaScript Object Notation)– A hierarchical format often used in APIs and web services. It allows for nested values, making it useful for representing complex, structured data such as Reddit posts or metadata-rich archives. JSON data often requires flattening before analysis in tools like R.

- XML (eXtensible Markup Language) – A tree-structured markup language designed to store richly encoded text and metadata. XML is widely used in scholarly editions and historical datasets (e.g., Old Bailey) and supports deeply nested content like legal documents or manuscripts. It is verbose but highly descriptive. XML is more verbose than JSON, and its verbosity enables more granular control over how texts are described and interpreted—especially when fine distinctions between text types or historical layers matter.
- HTML (HyperText Markup Language) – The foundational format of web pages. When scraping websites, analysts parse HTML to extract specific content.
- PDF (Portable Document Format) – A fixed-layout format intended for visual presentation rather than structured analysis. A fixed layout means that the position of text, images, and other elements is preserved exactly as intended across devices and platforms, replicating the appearance of a printed page. PDFs are therefore ideal for human reading and archival purposes but challenging for computational analysis. Extracting text from PDFs—especially those with poor OCR (Optical Character Recognition) quality—can be error-prone and often requires additional cleaning. Still, they remain a common format for digitized historical texts.
- R data files (`.rds`, `.RData`) – R data files are used to store R objects, such as data frames, in a way that preserves their structure and attributes. These are an efficient means of storage and retrieval within the R environment. When using packages like `hansardr`, the analyst loads `.RData` files.

Each of these formats has implications for how data can be queried, cleaned, stored, and interpreted and understanding these distinctions equips researchers to navigate the data ecosystem more effectively.

Data Type

"Data type" refers to the kind of value a variable can hold in R, such as numbers, text, or logical values. A data type refers to the kind of individual data value — its atomic unit. It defines a given piece of data is – whether it can be added or subtracted, whether it has a TRUE/FALSE value, and so on.

Data types determine how an object should be interpreted and used by R. Data types may differ from how values appear to human analysts.

Consider, for example, the following code, where the value "42" is enclosed in quotation marks, which tells R to treat it as a "string" or "character," not as a number (a "numeric" object).

```
# Create a character string
# - typeof() returns the data type of an object
my_number <- "42"

typeof(my_number) # Display the data type
```

```
## [1] "character"
```

Because the the variable my_number is a character-type object, not a number-type object, we cannot add it to another number without producing an error.

```
# Add 2 to the numeric value
# - the + operator performs numeric addition
my_number + 2 # Display the result
```

```
## Error in my_number + 2: non-numeric argument to binary operator
```

However, we can use the base R function `as.numeric()` to tell R to treat my_number henceforth as a number, not a string.

```r
# Convert the character string to a numeric value
# - as.numeric() converts the character string to a number
my_number <- as.numeric(my_number)

# Add 2 to the numeric value
my_number + 2 # Display the result
```

```
## [1] 44
```

The functions `as.character()`, `as.date()`, and `as.numeric()` are all useful for converting datatypes from one type to the next. When working with `hansardr`, some of us have struggled with deep confusion in our inability to work with the year, until we realized that at some point the year had been converted to a string object, not a numeric or date object.

Analysts can check the type of data stored in a column using the `typeof()` function. For example, the following code returns "character" as the type for the "text" column in Hansard:

```r
# Load the 1800 Hansard dataset
# - data() loads the dataset into the R session
data("hansard_1800")

# Display the data type of the text column
# - typeof() returns the data type of an object
typeof(hansard_1800$text)
```

```
## [1] "character"
```

Throughout this book, we have primarily worked with character and integer data. For other types of historical analyses, however, we may encounter data types or values we have not addressed in detail. Below are some common data types in R:

Data Type	Description	Example Code	`typeof()` Output
`numeric`	Decimal numbers (default type for numbers with decimals)	`x <- 3.14`	`"double"`
`integer`	Whole numbers	`y <- 42`	`"integer"`
`character`	Text or string values	`z <- "Mr. Gladstone"`	`"character"`
`logical`	Boolean TRUE/FALSE values	`a <- TRUE`	`"logical"`
`factor`	Categorical data stored as integers with labels	`f <- factor(c("low", "high"))`	`"integer"`

The meta-awareness that an analyst needs to become fluent with data types is not that different from an appreciation of the work of different media in representation. Consider René Magritte's famous 1929 painting *The Treachery of Images*, which depicts a pipe with the caption "Ceci n'est pas une pipe"—French for "This is not a pipe."^



When observing *The Treachery of Images*, the viewer sees a pipe, despite the instructions emblazoned below the image. If an analyst stares at the image and puzzles for hours, they may still see just a pipe.

The caption serves to remind us that a representation is an object, different in its nature from the original object represented; the pipe that Magritte held in his hand and painted is different from the picture of the pipe he made.

Just so, a human analyst may stare at their code for hours while asking themselves why two seemingly identical objects are behaving differently once they run their code. The painting draws our attention away from its content (a pipe) back to the pieces making up the content (oil and canvas). Programming, too, asks us to practice shifting our attention away from the content of our constructions ("42") and onto the components that make up our constructions (the rules that govern data types within a programing language).

When Data Type is Missing: NA and NULL Values

Not all values have a data type. Analysts delving deeper into programming may encounter values like `NA` and `NULL` often appear in datasets with missing records. Engaging datasets with `NA` and `NULL` values can be confusing because they analyst cannot treat them the same as data that have a type, like character or integer data.

In R, `NA` (Not Available) represents missing or undefined data. It used when a value is expected but not present, much like a blank cell in a spreadsheet.

In contrast, `NULL` indicates the absence of an object altogether, meaning the data structure or value does not exist at all.

Here is just one example of how `NA`'s are treated differently than character values. This example is an illustrative, fake historical snippet of a dataset of individuals recorded in an 1850 census table or notarial register, where some people have a listed occupation while others have a blank entry (`NA`). You want to count how many people were "laborers." If you treat NA as a string instead of a missing value, your counts will be wrong, because `NA` is the absence of information, not the word "laborer" misspelled or omitted.

The "occupation" column (which records the species' conservation status—e.g., endangered, threatened, or of least concern) contains character strings as well as `NA` values. As we will demonstrate, missing values (`NA`) behave yield a different result than text when undergoing string operations.

```
# Create a small census dataset
# - tibble() creates a data frame
# - c() combines multiple values into a column
```

```
census <- tibble(name = c("Mary Jenkins", "Samuel Price", "Hannah Lee", "Charles Morton",
                          "Unnamed Child"), # Record each person's name
                 occupation = c("washerwoman", NA, "seamstress", "laborer", NA), # Record each occupati
                 age = c(32, 44, 29, 51, 1)) # Record each person's age

census
```

```
## # A tibble: 5 x 3
##   name           occupation    age
##   <chr>          <chr>       <dbl>
## 1 Mary Jenkins   washerwoman    32
## 2 Samuel Price   <NA>           44
## 3 Hannah Lee     seamstress     29
## 4 Charles Morton laborer        51
## 5 Unnamed Child  <NA>            1
```

From printing just the first rows, multiple `NA`s appear.

We can now process the dataset and observe what happens when we try to implement functions that expect a string for the value of dataset rows. Here, we use `str_detect()` to identify whether or not a value is equal to the word "domesticated." We return our results in a new column named "is_washerwoman."

```
# Identify washerwomen in the census
# - mutate() creates a new column
# - str_detect() checks whether the occupation contains "washerwoman"
census %>%
  mutate(is_washerwoman = str_detect(occupation, "washerwoman")) # Create a TRUE/FALSE indicator
```

```
## # A tibble: 5 x 4
##   name           occupation    age is_washerwoman
##   <chr>          <chr>       <dbl> <lgl>
## 1 Mary Jenkins   washerwoman    32 TRUE
## 2 Samuel Price   <NA>           44 NA
## 3 Hannah Lee     seamstress     29 FALSE
## 4 Charles Morton laborer        51 FALSE
## 5 Unnamed Child  <NA>            1 NA
```

`str_detect()` returned `TRUE` if the value in the row is equal to "domesticated" and `FALSE` if it is not. However, multiple rows are left with `NA`, even though the values of those rows were not equal to "domesticated." `NA` is a different data type than a string, and so `str_detect()`, a function that anticipates a string, did not assign a true or false value.

We must use a different approach to identifying `NA` values:

```
# Identify rows with missing occupations
# - mutate() creates a new column
# - is.na() checks whether the occupation is missing
census %>%
  mutate(is_missing = is.na(occupation)) # Create a TRUE/FALSE indicator
```

```
## # A tibble: 5 x 4
##   name           occupation    age is_missing
```

```
##   <chr>          <chr>       <dbl> <lgl>
## 1 Mary Jenkins   washerwoman    32 FALSE
## 2 Samuel Price   <NA>           44 TRUE
## 3 Hannah Lee     seamstress     29 FALSE
## 4 Charles Morton laborer        51 FALSE
## 5 Unnamed Child  <NA>            1 TRUE
```

If we wish for the `NA` values to be treated like strings–which could be the case if we are analyzing historical records where some individuals' occupations are missing, and we want to treat "unknown" as a searchable category–we could replace them with the word "missing" or with an empty string (`""`). The following code provides an example for how we can accomplish treating treating `NA`s as a string:

```
# Replace missing occupations with the word "missing"
# - mutate() updates the occupation column
# - replace_na() replaces missing values with "missing"
census <- census %>%
  mutate(occupation = replace_na(occupation, "missing"))

# Display the first ten rows of the updated dataset
# - slice() keeps the specified rows
census %>%
  slice(1:10) # Keep the first ten rows
```

```
## # A tibble: 5 x 3
##   name           occupation    age
##   <chr>          <chr>       <dbl>
## 1 Mary Jenkins   washerwoman    32
## 2 Samuel Price   missing        44
## 3 Hannah Lee     seamstress     29
## 4 Charles Morton laborer        51
## 5 Unnamed Child  missing         1
```

Mixed Methods: Lists as Data Type

Lists are an important data type in R that also function as a data structure (defined in the following section). Lists are more complex than other types because they can include multiple data types within them.

Here, we define a list of continent names followed by their approximate population in millions.

```
# Create a list containing continent names and populations
# - list() stores objects of different types in a single container
continents_population <- list('South America', 644.54, 'Africa', 1305, 'Europe', 745.64, 'Asia', 4587)
```

```
# Show the data type of the object
# - typeof() returns the data type of an object
typeof(continents_population)
```

```
## [1] "list"
```

`continents_population` is the list type. But each item inside the list also has its own type. To inspect the data type, we can loop through the list and print each item's type like so:

```
# Display the data type of each item in the list
# - for() loops over each item in the list
# - typeof() returns the data type of the current item
# - print() displays the result
for (item in continents_population) {
  print(typeof(item)) }
```

```
## [1] "character"
## [1] "double"
## [1] "character"
## [1] "double"
## [1] "character"
## [1] "double"
## [1] "character"
## [1] "double"
```

Lists can contain varying types, which makes them well-suited for representing complex data. They can provide a more memory-efficient alternative to data frames, particularly in applications such as text mining large datasets. However, the flexibility of lists introduces additional complexity in terms of data types. If these complexities are not carefully managed, code may execute without error but yield unintended results due to discrepancies in type.

Data Structure

A data structure is a way of organizing and storing data so it can be accessed and modified. If a data type defines what a given piece of data is, a data structure is a container — it organizes multiple values (often of different types) in a structured way. It defines how data is stored, related, and accessed.

Throughout this book, we have consistently employed data frames, and more specifically tibbles, due to their easy integration with the `tidyverse` and their predictable behavior across a wide range of operations beyond the `tidyverse`. We have used intuitive naming conventions and pipeable workflows – all of which are customary for the main software package we have used, the `tidyverse`.[29]

The deliberate choice of using the `tidyverse` methods – and more specifically the dityverse data structure of tibbles =– for teaching minimizes the complications often introduced by less interoperable or idiosyncratic data structures.

But when an analyst begins text mining in earnest, they may soon venture into engaging other software packages in R that may be useful for text mining, but which use different data structures that can be confusing. As with data types, data structures determine the ways in which data can be accessed, transformed, and analyzed in R – so non-tidy formats may appear confusing and difficult-to-access to the analyst familiar only with the tools from the `tidyverse`.

For instance, the `quanteda` software package exports many outputs as S3 objects with custom print methods and slot-like components. The tm and topicmodels package export their outputs as S4 objects, some of which require knowledge of package-specific functions to extract results.[30]

Some common data structures in R that we have used for text mining are represented by the following table.

Table 9.4: Common R Data Structures in Text Mining

Structure	Dimension	Key Use for Text Mining	Example
Matrix	2D	Numeric structure for word embeddings or term-document counts	`matrix(rnorm(300), nrow = 100, ncol = 3)`
List	1D	Flexible storage for mixed or uniform data—e.g., texts, dates, names	`list(1800, "text", "author")`
Data Frame	2D	Standard tabular format for digitized records or structured text metadata	`data.frame(name = ..., date = ...)`
Tibble	2D	Tidyverse-friendly data frames optimized for large data and printing	`tibble(year, text)`
S3 Object	Varies	Used in packages like `quanteda`, `stm` with custom print/access behavior	`class(tokens_object) → "tokens"`
S4 Object	Varies	Formal, slot-based system used in `tm`, `topicmodels`, with accessor functions	`class(corpus_object) → "Corpus", "VCorpus"`

The solution to working with new data structures is to learn how to translate a new data structure into a format the analyst is familiar with.

So the goal is to: extract what you need → eeshape it → coerce it into a tibble

Working with the Output of Other R-Based Software: `Quanteda`

However, there are cases where a data frame may not be the most appropriate or efficient data structure. For example, when working with the `quanteda` package (such as when we performed Key Words in Context (KWIC), we rely on its custom-built data structure known as a `quanteda` "tokens object." The tokens object was specifically designed to optimize performance when handling large-scale text data within the `quanteda` ecosystem. In fact, it would be very difficult to perform some of the analyses we did without invoking the `quanteda` ecosystem given how efficient `quanteda` is at processing.

```
# Load the 1800 Hansard dataset
# - data() loads the dataset into the R session
data("hansard_1800")

# Extract the speech text
# - $ selects the text column from the dataset
texts <- hansard_1800$text

# Tokenize the speech text
# - tokens() converts the text into a quanteda tokens object
tokens_hansard <- tokens(texts)

# Search for the word "corn" in context
# - kwic() finds each occurrence of the keyword
# - window specifies the number of words shown before and after the keyword
kwic_result <- kwic(tokens_hansard,
                    pattern = "corn", # Search for the word "corn"
                    window = 5) # Show five words before and after each match
```

Impressively, the results from running `KWIC()` on an entire decade of the Hansard debates are now ready to analyze.

```r
# Display the first ten KWIC results
# - head() keeps the first ten results
# - kable() formats the results as a table
kwic_result %>%
  head(10) %>% # Keep the first ten results
  kable(caption = "KWIC results") # Format the results as a table
```

Table 9.5: KWIC results

docname	from	to	pre	keyword	post	pattern
text560	34	34	to the practice of hoarding	corn	in this of scarcity ,	corn
text1124	23	23	paid to foreign countries for	corn	and other articles of provision	corn
text1283	58	58	Mutiny Bills , the Seed	Corn	Exportation , the Greenland Whale	corn
text2284	11	11	and encouraging the importation of	corn	the act for regulating the	corn
text2358	33	33	all the last regulations respecting	corn	, permitting the importation ,	corn
text3349	3	3	The Irish	Corn	, Potatoe , and Provision	corn
text3449	25	25	to regulate the sale of	corn	in the City of Dublin	corn
text10135	29	29	Britain or Ireland , respecting	corn	, meal , or malt	corn
text11438	46	46	, and the bag of	corn	he principal merit of the	corn
text13829	26	26	help pf a feed of	corn	, it might not again	corn

However, attempting to use a `tidyverse` function on the `quanteda` tokens object returns an error.

```r
# Show the first five tokenized documents
# - slice() keeps the specified documents from the tokens object
slice(tokens_hansard, 1:5)
```

```
Error in UseMethod("slice"): no applicable method for 'slice' applied to an
object of class "tokens".
```

While `quanteda`'s design improves efficiency, it also introduces certain limitations. In particular, a `quanteda` tokens object cannot be passed directly to most base R or `tidyverse` functions. Instead, handling a `quanteda` obejct largely restricts us to using `quanteda` functions–or other functions explicitly designed to interface with this structure–unless we first convert the corpus back into a data frame.

To use `tidyverse` functions on the data we processed using `quanteda`, we must convert the data (in this case, the tokens object) to a tibble or data frame.

```r
# Convert the tokens object into a data frame
# - names() extracts the document IDs
# - lengths() counts the number of tokens in each document
# - rep() repeats each document ID for every token it contains
# - unlist() converts the tokens object into a character vector
# - tibble() stores the document IDs and tokens in a data frame
tokens_tibble <- tibble(doc_id = rep(names(tokens_hansard), lengths(tokens_hansard)), # Repeat each doc
                        token = unlist(tokens_hansard, use.names = FALSE)) # Store the individual token
```

Now we can use `tidyverse` functions on our data.

```r
# Show the first five rows of the token data frame
# - slice() keeps the specified rows
slice(tokens_tibble, 1:5)
```

```
## # A tibble: 5 x 2
##   doc_id token
##   <chr>  <chr>
## 1 text1  moved
## 2 text1  that
## 3 text1  Lord
## 4 text1  Walsingham
## 5 text1  be
```

Conclusion

No single book can prepare the analyst for all possible variations of data they may encounter, but the major kinds of data used as of the writing of this book are encompassed by the the methods we have set out here – using API calls and RSS feeds, bulk downloads, and pdf's; cleaning data, and paying attention to data type and structure. With practice, a data analyst may soon acquire the fluency with these methods needed to wrangle data from across the web and to engage with multiple software packages, each of which have their strengths and weaknesses.

Exercise

1. Why do scholars sometimes build their own archives instead of relying solely on canonical texts or curated corpora?
2. Explain the differences between data format, data type, and data structure, and discuss how misunderstandings about these distinctions could lead to errors or misinterpretations in computational analysis.
3. When used for historical research, why isn't data cleaning a purely technical task but an interpretive one?
4. Large language models are exceptionally handy tools, but they come with their own risks. What risks are associated with using large language models directly to clean text?
5. Find a dataset online. What is the source of your dataset? Is it from an API, bulk download, RSS feed, PDF, or scraped webpage? Why did you choose this source?

If your dataset comes from an API: - What endpoint are you using? - What parameters control the size or scope of the data returned? - Are you encountering rate limits?

If your dataset is a bulk download: - What format is it in (CSV, JSON, XML, etc.)? - Is it compressed? - How large is it?

If you scraped your data: - What HTML elements did you target?

6. Trace the provenance of your data. Who created it? For what purpose? What decisions were made during collection? What is missing? Write a short paragraph describing how the dataset's origin story might shape the conclusions drawn from it.
7. In the dataset you have identified, what counts as messy? Be specific.